

Avaliação Somativa

EventPulse - Gestão de Eventos Acadêmicos. A atividade possui as seguintes exigências técnicas: manipulação de estados complexos, persistência com localStorage, filtros dinâmicos, estilização condicional, correção de sintaxe/erros comuns de digitação em códigos base, e uma lista de desafios para que você implemente.

Objetivo

Desenvolver uma aplicação React que gerencie o cronograma de eventos acadêmicos, permitindo a categorização por tipo de atividade (Palestra, Workshop, Pannel), filtragem por status (Agendado, Em Andamento, Encerrado) e persistência de dados.

Requisitos Técnicos

- **Gerenciamento de Estado:** Uso de objetos complexos e múltiplos estados estruturados no useState.
- **Hooks de Efeito:** Sincronização e hidratação de dados com o localStorage.
- **Lógica de Filtro:** Implementação de visualizações parciais da lista original com base em categorias.
- **Estilização Condicional:** Alteração dinâmica de cores de borda e opacidade baseada nas propriedades do objeto (Tipo do Evento e Status).

1. Implementação do Código Base (App.jsx)

JavaScript

```
import React, { useState, useEffect } from 'react';
```

```
import './App.css';
```

```
function App() {
```

```
  const [eventTitle, setEventTitle] = useState("");
```

```
  const [eventType, setEventType] = useState("Palestra");
```

```
  const [eventList, setEventList] = useState([]);
```

```
  const [filter, setFilter] = useState("Todos");
```

```
// Carregar dados iniciais do LocalStorage

useEffect(() => {

  const savedEvents = localStorage.getItem("@eventpulse_data");

  if (savedEvents) setEventList(JSON.parse(savedEvents));

}, []);


// Sincronizar alterações com o LocalStorage

useEffect(() => {

  localStorage.setItem("@eventpulse_data", JSON.stringify(eventList));

}, [eventList]);


const addEvent = (e) => {

  e.preventDefault();

  if (!eventTitle.trim()) return;

  const newEvent = {

    id: crypto.randomUUID(),

    title: eventTitle,

    type: eventType,

    status: "Agendado", // Status inicial padrão

    date: new Date().toLocaleDateString()

  };

  setEventList([newEvent, ...eventList]);

  setEventTitle("");

};


const toggleStatus = (id) => {
```

```

setEventList(eventList.map(evt => {
  if (evt.id === id) {
    // Rotaciona o status do evento sequencialmente
    const nextStatus = evt.status === "Agendado" ? "Em Andamento" :
      evt.status === "Em Andamento" ? "Encerrado" : "Agendado";
    return { ...evt, status: nextStatus };
  }
  return evt;
}));
};

```

```

const deleteEvent = (id) => {
  setEventList(eventList.filter(evt => evt.id !== id));
};

```

```

const filteredEvents = eventList.filter(evt => {
  if (filter === "Agendados") return evt.status === "Agendado";
  if (filter === "Em Andamento") return evt.status === "Em Andamento";
  if (filter === "Encerrados") return evt.status === "Encerrado";
  return true;
});

```

```

return (
  <div className="app-container">
    <header>
      <h1>EventPulse</h1>
      <p>Gestão de Eventos Acadêmicos</p>
    </header>

```

```
<section className="form-section">

  <form onSubmit={addEvent}>

    <input

      value={eventTitle}

      onChange={(e) => setEventTitle(e.target.value)}

      placeholder="Nome do evento ou atividade..."

    />

    <select value={eventType} onChange={(e) => setEventType(e.target.value)}>

      <option value="Palestra">Palestra</option>

      <option value="Workshop">Workshop</option>

      <option value="Painel">Painel</option>

    </select>

    <button type="submit">Agendar</button>

  </form>

</section>
```

```
<section className="filter-section">

  [{"Todos", "Agendados", "Em Andamento", "Encerrados"].map(f => (

    <button

      key={f}

      className={filter === f ? "active" : ""}

      onClick={() => setFilter(f)}

    >

      {f}

    </button>

  )))

</section>
```

```

<main className="event-grid">

  {filteredEvents.map(item => (

    <div

      key={item.id}

      className={`event-card ${item.type.toLowerCase()}
${item.status.toLowerCase().replace(" ", "-")}`}

    >

      <div className="event-content">

        <h3>{item.title}</h3>

        <span className="event-tag">Tipo: {item.type}</span>

        <span className="status-badge">Status: {item.status}</span>

        <small>Registrado em: {item.date}</small>

      </div>

      <div className="event-actions">

        <button onClick={() => toggleStatus(item.id)} className="status-btn">

          {item.status === "Agendado" ? "Iniciar" : item.status === "Em Andamento"
? "Encerrar" : "Reiniciar"}

        </button>

        <button onClick={() => deleteEvent(item.id)} className="delete">

          Remover

        </button>

      </div>

    </div>

  )})

</main>

</div>

);

}

```

```
export default App;
```

2. Estilização Estruturada (App.css)

CSS

```
:root {  
  --palestra: #0284c7; /* Azul */  
  --workshop: #f59e0b; /* Âmbar/Amarelo */  
  --painel: #10b981; /* Verde */  
  --bg-encerrado: #e5e7eb;  
}  
  
.app-container {  
  max-width: 900px;  
  margin: 2rem auto;  
  padding: 0 1rem;  
  font-family: system-ui, -apple-system, sans-serif;  
  color: #1f2937;  
}  
  
header {  
  text-align: center;  
  margin-bottom: 2.5rem;  
}  
  
header h1 {  
  font-size: 2.5rem;  
  margin: 0;
```

```
color: #111827;  
}
```

```
.form-section form {  
  display: flex;  
  gap: 12px;  
  background: #f3f4f6;  
  padding: 20px;  
  border-radius: 10px;  
  box-shadow: 0 1px 3px rgba(0,0,0,0.05);  
}
```

```
.form-section input {  
  flex: 3;  
  padding: 12px;  
  border: 1px solid #d1d5db;  
  border-radius: 6px;  
  font-size: 1rem;  
}
```

```
.form-section select {  
  flex: 1;  
  padding: 12px;  
  border: 1px solid #d1d5db;  
  border-radius: 6px;  
  font-size: 1rem;  
  background: white;  
}
```

```
.form-section button {  
  padding: 12px 24px;  
  background: #4f46e5;  
  color: white;  
  border: none;  
  border-radius: 6px;  
  font-weight: 600;  
  cursor: pointer;  
}
```

```
.filter-section {  
  display: flex;  
  justify-content: center;  
  gap: 8px;  
  margin: 24px 0;  
}
```

```
.filter-section button {  
  padding: 8px 16px;  
  background: #f3f4f6;  
  border: 1px solid #e5e7eb;  
  border-radius: 20px;  
  cursor: pointer;  
  transition: all 0.2s;  
}
```

```
.filter-section button.active {
```



```
background: #111827;
color: white;
border-color: #111827;
}
```

```
.event-grid {
display: grid;
grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
gap: 20px;
}
```

```
.event-card {
padding: 18px;
border-radius: 8px;
border-top: 6px solid #ccc; /* Fallback */
background: white;
box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
display: flex;
flex-direction: column;
justify-content: space-between;
min-height: 180px;
}
```

```
/* Estilização Condicional por Tipo (Borda Superior) */
.event-card.palestra { border-top-color: var(--palestra); }
.event-card.workshop { border-top-color: var(--workshop); }
.event-card.painel { border-top-color: var(--painel); }
```

```
/* Estilização Condicional por Status */
```

```
.event-card.em-andamento {  
  animation: pulse border 2s infinite alternate;  
  background: #fbfbfe;  
}
```

```
.event-card.encerrado {  
  opacity: 0.55;  
  background: var(--bg-encerrado);  
  box-shadow: none;  
}
```

```
.event-content h3 {  
  margin: 0 0 12px 0;  
  font-size: 1.2rem;  
}
```

```
.event-tag, .status-badge {  
  display: block;  
  font-size: 0.85rem;  
  margin-bottom: 4px;  
}
```

```
.event-tag { font-weight: 500; color: #4b5563; }  
.status-badge { font-weight: bold; color: #1f2937; }
```

```
small {  
  color: #9ca3af;
```

```
display: block;
margin-top: 8px;
}
```

```
.event-actions {
margin-top: 18px;
display: flex;
justify-content: space-between;
gap: 8px;
}
```

```
.event-actions button {
flex: 1;
padding: 8px;
font-size: 0.9rem;
border: 1px solid #d1d5db;
border-radius: 4px;
cursor: pointer;
}
```

```
.event-actions button.status-btn {
background: #f9fafb;
font-weight: 500;
}
```

```
.event-actions button.delete {
background: #ef4444;
color: white;
```

```
border-color: #ef4444;
```

```
}
```

O que você como aluno expert e técnico em dev deve realizar de implementações nesta somativa??

Para expandir a regra de negócios e simular um cenário real de entrega de software de gerenciamento acadêmico, você deve implementar as seguintes features adicionais:

1. **Destaque Cronológico de Workshops:** Crie uma lógica que force todos os itens do tipo "Workshop" a ficarem fixados no início da listagem renderizada, independentemente da ordem em que foram adicionados.
2. **Filtro por Caixa de Pesquisa:** Adicione um input de texto acima da grid de eventos. A listagem deve ser filtrada em tempo real (caractere por caractere) comparando o termo digitado com o título (title) do evento.
3. **Vagas Disponíveis (Novo Estado interno):** Adicione uma propriedade numérica chamada vagas ao criar o evento (o usuário escolhe no formulário entre 10, 30 ou 50 vagas). Na área de ações do card do evento, implemente um botão "Inscrever Aluno" que diminui o número de vagas disponíveis em 1 a cada clique. Se chegar a 0, troque o texto do botão para "Esgotado" e o desabilite.
4. **Alerta Preventivo de Limpeza:** Implementar um botão geral no cabeçalho chamado "Limpar Cronograma". Antes de apagar todo o localStorage e esvaziar o estado eventList, a aplicação precisa exibir um diálogo nativo do navegador (window.confirm) validando a ação do usuário.
5. **Estilização:** Faça ao menos 3 alterações indeléveis na estrutura do CCS acima, que seja marcante ao site. Crie um botão (redondo) com o favicon em anexo no canto inferior direito da página que, ao ser clicado exiba um modal listando todas essas alterações.